Eligify System Architecture & Working Report

Subject: Subsystem breakdown, technology stack, and engineering rationale for the privacy-preserving national welfare platform.

Compliance: Digital Personal Data Protection (DPDP) Act 2023 • UIDAI Aadhaar Data Minimization Standards

1. Executive Architecture Overview

Eligify determines citizen entitlements across **3,400+ Central and State welfare schemes** using a zero-knowledge, deterministic rule engine. The architecture solves three core challenges in public welfare delivery:

- **Data Minimization & Confidentiality:** No raw document photos are ever saved. Sensitive demographics are encrypted client-side using non-extractable keys.
- **Shared Device Isolation:** Accounts on shared village kiosks (CSCs) are strictly isolated with in-memory sessions and zero cross-user credential exposure.
- **Deterministic Entitlement Accuracy:** Rule evaluation is fully explainable with exact qualification criteria and official portal links.

2. Step 1: User Authentication & Zero-Knowledge Vault

What is used: Web Crypto API (SubtleCrypto), PBKDF2-SHA256 (600,000 iterations), HKDF (RFC 5869), AES-256-GCM, IndexedDB.

Why it is used:

- **Client-Side Cryptography:** The master password never travels over the network. The server only sees an ephemeral `loginProof` derived via HKDF.
- **Envelope Encryption:** A non-extractable 256-bit AES-GCM Data Encryption Key (DEK) encrypts user profile blobs. The DEK itself is wrapped with a Key Encryption Key (KEK) derived from the user's password.

- **Emergency Recovery:** A 32-character Crockford Base32 recovery key is generated at registration. It derives a separate recovery KEK, allowing citizens to recover their account without backdoors.

3. Step 2: User Isolation on Shared Devices

What is used: RAM-only session management (`src/services/session.js`), 5-minute idle activity listeners, AAD ciphertext binding, `Cache-Control: no-store`.

Why it is used:

- **No Account Switcher / Usernames Exposed:** The login screen has no dropdown listing prior users. Each citizen types their own credentials in a private input.
- **RAM-Only Lifecycle:** The decrypted data and encryption key exist solely in volatile memory. On logout or after 5 minutes of inactivity, memory is cleared.
- **AAD Binding (Additional Authenticated Data):** AES-GCM ciphertexts are cryptographically bound to the account alias (`data/v1/${alias}`). An attacker cannot swap ciphertexts between accounts.

4. Step 3: Database Layer & Row-Level Security (PostgreSQL & SQLite)

What is used: PostgreSQL 14+ with `FORCE ROW LEVEL SECURITY` (`001_postgres_rls.sql`), Node.js native SQLite in WAL mode (`backend/database.js`).

Why it is used:

- **PostgreSQL RLS Safety Net:** Enforces database-level isolation. Queries set `SET LOCAL app.current_user_id = $1`. Even if a backend query omits a WHERE clause, the PostgreSQL engine filters out all rows belonging to other users.
- **High-Speed Scheme Catalog:** 3,400 schemes with parsed rules, ministries, benefits, and portal links are stored in SQLite in WAL mode, returning filtered results in sub-millisecond times with zero external database configuration.

5. Step 4: Document Verification & Client-Side OCR

What is used: Tesseract.js 7.0 (WebWorker / Wasm), Lexical regex parser (`documentParser.js`).

Why it is used:

- **Zero-Storage DPDP Compliance:** Uploaded images are processed in the browser's sandbox memory. The raw image buffer is dropped immediately after OCR. No document

images are ever stored on disk or server.

- **Aadhaar Masking:** Full 12-digit Aadhaar numbers are never stored. Only the last 4 digits (`•••• •••• 3847`) are retained.
- **Anti-Collision Heuristics:** Income parsing utilizes lexical anchors to prevent confusing 6-digit postal PIN codes (e.g. 412207) or application numbers with annual income.

6. Step 5: Profile Verification & State Management

What is used: React 18 bidirectional synchronization hook (`useEffect`), numeric coercion validation.

Why it is used:

- **Human-in-the-Loop Validation:** Gives citizens full transparency to review, verify, and modify attributes extracted from documents before running eligibility calculations.
- **Zero-Value Support:** Preserves `0` values for landless laborers (0 acres) and unemployed individuals (₹0 income) without falling back to mock placeholders.

7. Step 6: Deterministic Eligibility Rule Engine

What is used: Dual-layer deterministic rule evaluator (mirrored in client and backend).

Why it is used:

- **Explainable Decisions:** Unlike generative AI, legal welfare qualification requires strict, reproducible evaluation. If a citizen is ineligible, the system outputs the exact failure reason (e.g. age ceiling or income limit).
- **Direct Application Links:** Every scheme card provides verified links directly to the official ministry application portals.

8. Verification & Test Suite Summary

Test Suite	Command	What It Validates	Result
User Isolation & RLS	`npm run test:isolation`	Cross-user DEK rejection, AAD binding, session wiping, Postgres RLS simulation	**100% Passed**
Zero-Knowledge Vault	`npm run test:vault`	14 cryptographic tests: PBKDF2, non-extractable keys, recovery reset, tampering	**14/14 Passed**

Test Suite	Command	What It Validates	Result
Document Extraction	`npm run test:extraction`	Aadhaar, Income, Caste, Ration, Land parsing and profile merging	**100% Passed**
Rule Engine	`npm run test:rules`	Deterministic evaluation across 3,400 schemes with criteria verification	**100% Passed**

> **✓ System Status: Production Ready & Verified**
>
> All test suites pass cleanly, both directory repositories are synchronized, and the frontend builds with zero compilation errors.